

Algorithm Analysis and Python Concepts

1

For an unsorted list, the worst-case time complexity to find the k th smallest value is $O(n)$ using the median-of-medians algorithm. If the list is already sorted, the time complexity is $O(1)$ because the element can be directly accessed at index $k-1$.

2

```
def binary_search_first(arr, target):
    low, high = 0, len(arr) - 1
    result = -1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == target:
            result = mid
            high = mid - 1
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return result
```

This modified algorithm searches the left half of the array even after finding the target. It halves the search space during every iteration until the pointers cross, guaranteeing an $O(\log n)$ run time.

3

```
def find_negatives(lst):
    return [x for x in lst if x < 0]
```

The worst-case scenario occurs on every execution because the function must check every single element in the list to determine if it is negative. The run time for this case is always $O(n)$.

4

Initial List: 80, 7, 24, 16, 43, 91, 35, 2, 19, 72

Bubble Sort

1: 7, 24, 16, 43, 80, 35, 2, 19, 72, 91
2: 7, 16, 24, 43, 35, 2, 19, 72, 80, 91
3: 7, 16, 24, 35, 2, 19, 43, 72, 80, 91
4: 7, 16, 24, 2, 19, 35, 43, 72, 80, 91
5: 7, 16, 2, 19, 24, 35, 43, 72, 80, 91
6: 7, 2, 16, 19, 24, 35, 43, 72, 80, 91
7: 2, 7, 16, 19, 24, 35, 43, 72, 80, 91

8: 2, 7, 16, 19, 24, 35, 43, 72, 80, 91

9: 2, 7, 16, 19, 24, 35, 43, 72, 80, 91

Selection Sort

1: 2, 7, 24, 16, 43, 91, 35, 80, 19, 72

2: 2, 7, 24, 16, 43, 91, 35, 80, 19, 72

3: 2, 7, 16, 24, 43, 91, 35, 80, 19, 72

4: 2, 7, 16, 19, 43, 91, 35, 80, 24, 72

5: 2, 7, 16, 19, 24, 91, 35, 80, 43, 72

6: 2, 7, 16, 19, 24, 35, 91, 80, 43, 72

7: 2, 7, 16, 19, 24, 35, 43, 80, 91, 72

8: 2, 7, 16, 19, 24, 35, 43, 72, 91, 80

9: 2, 7, 16, 19, 24, 35, 43, 72, 80, 91

Insertion Sort

1: 7, 80, 24, 16, 43, 91, 35, 2, 19, 72

2: 7, 24, 80, 16, 43, 91, 35, 2, 19, 72

3: 7, 16, 24, 80, 43, 91, 35, 2, 19, 72

4: 7, 16, 24, 43, 80, 91, 35, 2, 19, 72

5: 7, 16, 24, 43, 80, 91, 35, 2, 19, 72

6: 7, 16, 24, 35, 43, 80, 91, 2, 19, 72

7: 2, 7, 16, 24, 35, 43, 80, 91, 19, 72

8: 2, 7, 16, 19, 24, 35, 43, 80, 91, 72

9: 2, 7, 16, 19, 24, 35, 43, 72, 80, 91

5

The best-case time complexity is $O(n)$, which occurs when the list is already sorted and requires only one comparison per element. The worst-case time complexity is $O(n^2)$, which occurs when the list is in reverse order and requires maximum comparisons and shifts.

6

```
dataset = [[1, 2], [3, 4], [5, 6]]
total_sum = sum(item for row in dataset for item in row)
```

This comprehension iterates over every row in the dataset and then over every item in that row, passing all numbers to the built-in sum function.